Table of Contents

1. Executive Summary

MOXI AI system audit covering 8+17+7+4+2+6 domains across 44 files

This comprehensive audit examines the MOXI AI system, VIXOR's flagship artificial intelligence architecture that powers the platform's trading intelligence capabilities. MOXI is designed as a unified AI companion that combines the roles of analyst, hunter, coach, and governor into a single proactive entity. The system spans 44 files across six major subsystems: the core MOXI domain (8 files), the copilot domain with multi-agent orchestration (17 files), the debate engine for multi-perspective analysis (7 files), the LLM routing infrastructure supporting four providers (6 files), the user memory system (2 files), the tool registry (5 files), and the notification hub (6 files). This audit evaluates every component across architecture quality, implementation completeness, scalability, and alignment with the AI Operating System product vision.

The MOXI system represents a bold and architecturally ambitious attempt to build an AI-first trading companion. The core concept of a unified AI entity that can analyze markets, track signals, manage risk, and coach traders simultaneously is both the system's greatest strength and its most significant challenge. The current implementation delivers a functional chat-based AI experience with tool execution, proactive notifications, persona customization, and multi-agent consensus capabilities. However, several critical gaps prevent it from achieving the "AI Operating System" vision described in the product roadmap. Key deficiencies include the absence of conversation persistence (every request starts fresh), no semantic memory beyond simple key-value storage, a purely keyword-based intent detection system with no LLM-powered understanding, hardcoded mock implementations for the four specialized agents (Coach, Analyst, Governor, Hunter), and a debate engine that uses deterministic voting rather than actual LLM deliberation.

Despite these limitations, the architectural foundation is remarkably solid. The domain-driven design cleanly separates concerns, the LLM router with automatic fallback across four providers (ZAI, Anthropic, Groq, OpenAI) ensures resilience, the context engine demonstrates excellent parallel data aggregation patterns, and the notification hub implements genuinely useful proactive intelligence. The system scores an overall 5.2 out of 10, reflecting strong architectural bones but incomplete AI muscle. With focused investment in the ten P0 tasks identified in Chapter 14, MOXI has the potential to evolve from a capable chatbot into a genuinely intelligent trading partner that remembers, learns, reasons, and acts autonomously on behalf of its users.

1.1 Overall System Scores

Ove *W*	Arch *Dom*	A *Intent*	Us *Cha*	 *Conte*	C *Clear*	**5.0** Product Maturity *Functional but incomplete*

1.2 Key Findings Summary

The audit identified 47 distinct problems across the MOXI ecosystem. Of these, 14 are classified as P0 (Critical), meaning they directly block the AI Operating System vision or represent fundamental architectural gaps. 18 are P1 (High Priority), representing significant quality and capability improvements. The remaining 15 are P2 (Medium), covering advanced features and optimizations. The most critical finding is that the four "AI agents" in the copilot domain (Coach, Analyst, Governor, Hunter) are hardcoded mock implementations that return static or random responses with no actual intelligence. Meanwhile, the debate engine uses deterministic heuristic voting rather than real LLM-powered deliberation, making its outputs predictable and limited. The memory system, while functional, operates on simple key-value pairs with no semantic search capability, limiting MOXI's ability to form deep contextual understanding of individual traders.

2. AI Product Vision

Scoring MOXI across 5 product dimensions

MOXI's product vision positions it as an "AI Operating System for Traders" rather than a simple chatbot or signal generator. This vision encompasses five core dimensions: Proactive Intelligence (the AI independently identifies opportunities and risks without being asked), Deep Personalization (the system builds a rich model of each trader's style, preferences, mistakes, and goals), Multi-Agent Reasoning (multiple specialized AI perspectives collaborate to produce superior analysis), Actionable Autonomy (the AI can execute actions on behalf of the trader within guardrails), and Continuous Learning (the system improves over time by observing outcomes and adapting). This chapter evaluates the current implementation against each dimension, providing both a score and a detailed gap analysis that informs the improvement roadmap in subsequent chapters.

The strongest dimension is Proactive Intelligence, where the notification hub already delivers genuine value by detecting overexposure, signal proximity to targets, and upcoming high-impact economic events. This is the closest the system comes to the "AI OS" vision because MOXI does things the user did not explicitly ask for, which is the defining characteristic of proactive intelligence. The weakest dimension is Continuous Learning, where the memory system stores simple key-value pairs and there is no outcome tracking, no prompt experimentation framework, and no feedback loop that improves AI responses based on measured quality. Between these extremes, Deep Personalization shows promise through the persona system and memory categories, but lacks semantic depth. Multi-Agent Reasoning exists conceptually but is undermined by mock implementations. Actionable Autonomy is limited to predefined tool calls with no planning or multi-step execution capability.

2.1 Vision Dimension Scores

6.	4.	3.	4.	3.0
Proactive In	Deep Perso	Multi-Agent	Actionable A	Continuous Learning
Notification	*K/V memory,*	*Mock agents, d*	*Tool execution,*	*No outcome tracking*

2.2 Proactive Intelligence (Score: 6.5/10)

The notification hub in src/domains/moxi/notification-hub.ts represents the system's strongest alignment with the product vision. Three distinct proactive detectors are implemented: detectOverexposure()

analyzes correlated positions across active signal trackings to warn when a trader is overexposed to a single currency (e.g., three BUY signals on EUR pairs), detectSignalProximity() monitors the distance of active signals to their take-profit and stop-loss levels and generates alerts when price approaches within 10% of TP or 15% of SL, and detectEventRisk() cross-references the user's active positions with upcoming high-impact economic events to warn about catalysts that could disrupt open trades. These detectors are genuinely useful and demonstrate a clear understanding of what traders need but may not think to ask about. The gap is that these detectors use simple heuristic rules rather than learned patterns, and there are no detectors for regime changes, volatility spikes, unusual volume, or correlation breakdowns.

2.3 Deep Personalization (Score: 4.0/10)

Personalization has two components: the persona system in src/domains/moxi/persona.ts and the memory system in src/shared/memory/store.ts. The persona system allows users to customize MOXI's name, personality description, communication style (formal/casual/mixed), and avatar variant (8 options: default, bull, bear, crystal, flame, ocean, phantom, nova). This is a well-designed surface-level personalization layer. The memory system provides five categories (preference, behavior, mistake, insight, strategy) with confidence scores and source tracking. However, the memory is stored as simple JSON strings in PostgreSQL with no semantic indexing. The retrieve() method filters by category but cannot match by meaning, making it impossible for MOXI to recall relevant memories based on conversational context. There is no learning from conversation patterns, no adaptation of communication style over time, and no modeling of the trader's evolving skill level. The persona system lacks timezone awareness, language detection, and adaptive tone adjustment based on the trader's emotional state.

2.4 Multi-Agent Reasoning (Score: 3.5/10)

The multi-agent system exists in two separate implementations that are not unified. The first is the copilot domain (src/domains/copilot/server/) which defines four LLM-powered agents (Market Analyst, Risk Manager, News Analyst, Strategy Builder) with detailed system prompts and an orchestrator that supports single-agent, auto-selected, and consensus modes. These agents are genuinely intelligent because they use real LLM calls with rich context injection. The second implementation is the four "v4 agents" (Coach, Analyst, Governor, Hunter) in separate files (coach.agent.ts, analyst.agent.ts, governor.agent.ts, hunter.agent.ts) that are hardcoded mocks returning static or random responses with no LLM integration. The debate engine (src/domains/debate/engine/debate.engine.ts) uses these mock agents to produce deterministic voting outcomes. This dual implementation creates confusion and undermines credibility. Additionally, MOXI itself is described as a "unified companion" that combines all four agent capabilities, but this is implemented purely through prompt engineering rather than actual agent composition.

2.5 Actionable Autonomy (Score: 4.5/10)

MOXI can execute seven predefined tools (analyzePair, trackSignal, createPriceAlert, getMarketSummary, getPortfolio, getWatchlist, getSignalStatus) through a keyword-based intent detection system in copilot-agent.ts. The detection uses regex pattern matching against common trading phrases to identify user intent and extract parameters like trading pairs, conditions, and prices. When a match is found with sufficient parameters, the tool is dispatched through the ToolRouter to the ToolRegistry for execution. This works well for straightforward requests like "set an alert for BTC above 110000" but fails for ambiguous, complex, or multi-step requests. There is no planning capability, no tool composition, no conditional execution, and no ability to chain multiple tools into a workflow. The lack of native LLM function calling means all tool orchestration is done through brittle regex patterns rather than the model's own understanding.

2.6 Continuous Learning (Score: 3.0/10)

This is the weakest dimension. The memory system has a learn() method that increases confidence when the same observation is repeated, but this is rudimentary reinforcement of simple key-value pairs. There is no mechanism to track whether MOXI's suggestions were correct, no outcome logging for predictions, no feedback collection beyond a stub recordFeedback() function that only logs to console, no A/B testing framework for prompt variants, and no metrics on response quality or user satisfaction. The copilot domain has a feedback.ts file with a recordFeedback() function that accepts ratings and comments but only prints to console without storing anything. The experiment domain (src/domains/experiment/) has infrastructure for evolutionary prompt optimization but is not connected to the MOXI system. Without a closed feedback loop that measures outcomes and adjusts behavior, MOXI cannot fulfill the "continuous learning" dimension of its product vision. The memory TTL (time-to-live) is also absent, meaning stale preferences may persist indefinitely.

3. AI Architecture

17 engines analyzed: purpose, state, problems, architecture, and priority

The MOXI AI architecture is built on a domain-driven design pattern with clear separation of concerns. The system comprises seventeen distinct engines across six subsystems, each with a specific responsibility in the AI pipeline. This chapter examines each engine in detail, evaluating its implementation state, architectural quality, integration patterns, and identifying specific problems and missing capabilities. The engines are organized by subsystem: Core MOXI Domain (3 engines), Copilot Domain (5 engines), Debate Domain (2 engines), LLM Infrastructure (3 engines), Memory System (2 engines), and Tool/Notification Infrastructure (2 engines). Each engine is scored individually, and architectural patterns are analyzed for consistency and extensibility.

The overall architectural pattern follows a layered approach: raw data flows from Supabase and external APIs through the Context Engine into formatted context strings, which are injected into LLM system prompts via the Prompt Builder, and the LLM Router selects the appropriate provider and handles fallback. This pipeline is clean but has significant bottlenecks. The Context Engine fetches data in parallel (which is excellent) but produces large context strings that consume LLM tokens rapidly. The Prompt Builder has no token budget management, meaning complex contexts can exceed model limits or waste tokens on low-relevance data. The LLM Router is well-designed with automatic fallback across four providers but lacks per-request cost tracking, token counting, and model selection optimization. The lack of streaming support for MOXI's primary askMoxi endpoint means users wait for full responses rather than seeing progressive output.

3.1 Engine Inventory

ID	Engine	File	State	Purpose	Key Issue	Score
E01	Context Engine	moxi/context-engine.ts	Active	Assembles user context for prompts	Parallel data fetch, no token budget	6
E02	Prompt Builder	moxi/prompt.ts	Active	Builds system prompt from context	No token management, static format	5
E03	Notification Hub	moxi/notification-hub.ts	Active	Proactive insight detection	Heuristic only, 3 detectors	6
E04	Agent Orchestrator	copilot/server/agent-orchestrator.ts	Active	Routes to LLM agents, consensus	Well-designed, supports streaming	7
E05	Copilot Agent	copilot/server/copilot-agent.ts	Active	P1 tool-using intent layer	Regex-only intent, no LLM	5
E06	Coach Agent	copilot/server/coach.agent.ts	Mock	Trading psychology coaching	Hardcoded, no AI	2

ID	Engine	File	State	Purpose	Key Issue	Score
E07	Analyst Agent	copilot/server/analyst.agent.ts	Mock	Behavioral analysis reports	Static output, no AI	2
E08	Governor Agent	copilot/server/governor.agent.ts	Mock	Risk assessment engine	Simple math, no AI	2
E09	Hunter Agent	copilot/server/hunter.agent.ts	Mock	Smart money scoring	Random scores, no AI	1
E10	Debate Engine	debate/engine/debate.engine.ts	Active	Multi-agent voting synthesis	Deterministic, not LLM-powered	4
E11	LLM Router	shared/llm/router.ts	Active	Provider selection and fallback	4 providers, no cost tracking	7
E12	Memory Store	shared/memory/store.ts	Active	Long-term user memory	K/V only, no semantic search	5
E13	Memory Bootstrap	shared/memory/index.ts	Active	Memory module exports	Thin wrapper	8
E14	Tool Registry	shared/tool-registry/	Active	Tool schema and dispatch	Well-designed, no composition	6
E15	Tool Router	shared/tool-router/index.ts	Active	Routes tool calls to handlers	Clean dispatch pattern	7
E16	Notification Channels	shared/notifications/	Active	Multi-channel notification	4 channels, well-structured	6
E17	Decision Store	copilot/server/decision-store.ts	Stub	Stores AI decisions	Minimal implementation	3

3.2 Architecture Quality by Engine

The highest-scoring engines are the Agent Orchestrator (7/10), LLM Router (7/10), and Tool Router (7/10). These share common architectural strengths: clean separation of concerns, proper error handling with fallback strategies, lazy initialization of singletons, and well-defined interfaces. The Agent Orchestrator is particularly impressive with its support for single-agent, auto-selected, and consensus modes, plus streaming support. The LLM Router implements a priority chain with automatic fallback that handles configuration errors gracefully. The lowest-scoring engines are the four mock agents (1-2/10), which are placeholders that return hardcoded responses. The Hunter Agent is the worst offender, using Math.random() to generate scores between 20-90 based on string length, making its outputs meaningless. The Debate Engine scores only 4/10 because it uses deterministic heuristic voting (based on analysis result properties) rather than actual LLM-powered deliberation, producing predictable and non-insightful consensus outputs.

A critical architectural problem is the dual agent system. The copilot domain contains two completely separate agent frameworks: the LLM-powered agents (Market Analyst, Risk Manager, News Analyst, Strategy Builder) defined in agents.ts with detailed system prompts, and the mock v4 agents (Coach,

Analyst, Governor, Hunter) in separate files. These two systems have different type definitions, different interfaces, different calling conventions, and different capabilities. MOXI is positioned as a "unified companion" but is neither of these systems, instead having its own context engine and prompt builder. This fragmentation makes it extremely difficult to maintain, extend, or reason about the AI layer. Unifying these into a single agent framework is identified as task AI-008 and is one of the most critical architectural improvements needed.

3.3 Engine State Distribution

State	Count	Percentage	Description
Active (functional)	12	70%	Working engines with varying quality
Mock (hardcoded)	4	24%	Coach, Analyst, Governor, Hunter agents
Stub (placeholder)	1	6%	Decision Store - minimal implementation

4. User Memory System

Structured long-term memory: preference, behavior, mistake, insight, strategy

The memory system is implemented in src/shared/memory/store.ts as a PostgreSQL-backed structured memory store with five categories: preference (user trading preferences like preferred pairs and timeframes), behavior (observed patterns like trade frequency and session timing), mistake (recorded trading mistakes for learning), insight (AI-generated observations about the user), and strategy (trading strategy notes and rules). Each memory entry has a confidence score (0-1) representing the system's certainty about the observation, a source field tracking what generated the memory (copilot, user_action, system), and timestamps for creation and updates. The MemoryStore class provides store(), retrieve(), retrieveCategory(), retrieveAll(), forget(), and learn() methods, plus a contextForPrompt() method that formats all memories into a string suitable for injection into LLM system prompts.

The architecture is fundamentally sound in concept. The five-category taxonomy covers the most important aspects of a trader's profile. The confidence scoring and source tracking enable differentiation between high-confidence observations (repeated behaviors) and low-confidence ones (single occurrences). The learn() method implements a basic reinforcement pattern where repeated observations increase confidence to 1.0. The contextForPrompt() method properly formats memories for prompt injection, and the MOXI context engine already integrates memory context into its prompt building pipeline. However, the implementation has several critical limitations. First, the retrieve() method only filters by category and cannot match by semantic meaning. If MOXI stores a preference for "XAU/USD" and later needs to recall information about "gold," it will not find the match. Second, the upsert conflict constraint on (user_id, category, content) means the system can only store one memory per category per user, which is extremely limiting. A trader may have dozens of preferences, not just one. Third, there is no TTL mechanism, so stale memories persist indefinitely. Fourth, the forget() method deletes an entire category rather than a specific key, making granular memory management impossible.

4.1 Memory Architecture

The storage layer uses a user_memories table with columns for user_id, category (text), content (JSONB), metadata (JSONB with key/confidence/source), created_at, and updated_at. The metadata column stores the structured fields (key, confidence, source) as a JSON object, while the actual value is serialized into the content column. This design has a fundamental flaw: the upsert onConflict is set to (user_id, category, content), but the content is the serialized JSON value. This means if the value changes, a new row is created rather than updating the existing one. The retrieve() method filters by user_id and category and returns maybeSingle, which means it will only find the most recently inserted row for that category, silently losing any other memories in the same category. This is a P0 bug that effectively limits each memory category to a single entry per user.

The integration with MOXI is handled in the context-engine.ts file, which calls MemoryStore.contextForPrompt(userId) to retrieve formatted memories and injects them into the prompt via the formatMoxiContext function. The memory context is appended to the system prompt with a specific instruction: "IMPORTANT: Use this memory to personalize. Reference specific preferences and patterns." The copilot-agent.ts also uses memory through the learn() method, storing detected intents and queried pairs as behavioral observations. This creates a basic feedback loop where user interactions inform MOXI's understanding, but the loop is very shallow because it only records the last intent and queried pair without any deeper analysis of the interaction pattern.

4.2 Memory Categories and Gaps

Category	Description	State	Key Gap
preference	User preferences	Working	Single entry limit, no semantic match
behavior	Observed patterns	Working	Only last_intent and queried_pair tracked
mistake	Trading mistakes	Schema only	No automatic mistake detection or recording
insight	AI observations	Schema only	No code generates insight memories
strategy	Strategy notes	Schema only	No strategy extraction from conversations
emotion	Emotional state	Missing	No emotion detection or tracking system
session	Session context	Missing	No short-term conversational memory buffer
outcome	Prediction results	Missing	No outcome tracking or learning loop

4.3 Memory Problems

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MP-001	P0	Upsert constraint only stores one memory per category per user	Critical	High	shared/memory/store.ts	onConflict on (user_id, category, content) with content as JSON
MP-002	P0	No semantic search - cannot match by meaning	Critical	High	shared/memory/store.ts	Retrieve filters by category only, no embedding/vector
MP-003	P1	forget() deletes entire category instead of specific key	High	Medium	shared/memory/store.ts	DELETE WHERE user_id AND category, no key filter
MP-004	P1	No memory TTL or expiration mechanism	High	Medium	shared/memory/store.ts	No updated_at-based pruning logic

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MP-005	P1	contextForPrompt returns raw JSON strings	Medium	Low	shared/memory/store.ts	JSON.stringify in prompt reduces readability
MP-006	P2	No memory priority or relevance scoring	Medium	Low	shared/memory/store.ts	All memories treated equally regardless of recency

4.4 Memory System Score

5.0

Memory System

Structured but limited

5. Persona Engine

MOXI personality customization with 8 avatar variants

The persona engine in src/domains/moxi/persona.ts manages MOXI's personality configuration on a per-user basis. Each user can customize their MOXI instance through four dimensions: name (up to 30 characters), personality description (up to 500 characters defining the AI's character), communication style (formal, casual, or mixed), and avatar variant (one of 8 options). The default persona is defined in types.ts as a sharp, proactive AI trading companion that speaks directly in trader language, has a slight edge of humor, and is always data-driven. The avatar variants are purely visual: default (green-to-cyan gradient), bull (green, optimistic), bear (red, cautious), crystal (purple, analytical), flame (orange-red, aggressive), ocean (blue, calm), phantom (gray, stealth), and nova (amber-red, explosive). Each variant has a gradient, symbol, label, and description.

The persona system stores its configuration in the moxi_personas table with a user_id primary key and uses upsert for updates. The getMoxiPersona() function falls back gracefully to the default persona if no custom persona exists or if the database query fails. The updateMoxiPersona() function accepts partial updates and always sets is_customized to true. This is a clean, well-implemented CRUD system. However, the persona system has significant limitations in depth. The personality description is a single free-text field with no structure, making it impossible to systematically adjust MOXI's behavior based on persona attributes. The communication style toggle (formal/casual/mixed) only changes a single word in the system prompt ("professional and precise" vs "conversational and direct"), which is a superficial adjustment that does not meaningfully affect response tone, length, emoji usage, or formality level. There is no timezone awareness, no language detection, no adaptive tone based on conversation context, and no dynamic persona evolution based on accumulated user knowledge.

5.1 Persona Components

nponent	Type	Description	Limitation
e	String (30 char)	Display name for the AI	User sets once, static
onality	Text (500 char)	Core personality description	Free text, no structured attributes
munication	Enum (3)	formal/casual/mixed	Only affects one word in prompt
ar Variant	Enum (8)	Visual theme	Purely cosmetic, no AI behavior change
Token ID	Optional string	Links to NFT for avatar	Not implemented, schema only
stomized	Boolean	Whether user modified persona	Set on any update

5.2 Missing Persona Features

The persona system needs several additions to achieve meaningful personalization. First, adaptive persona adjustment based on accumulated memory: if the memory system knows a user prefers detailed analysis, MOXI should automatically adjust response verbosity without manual persona changes. Second, emotion-aware tone adjustment: when MOXI detects that a user is frustrated (e.g., recent losses, negative sentiment in messages), it should adopt a more supportive and cautious tone. Third, timezone-aware greetings and session timing: MOXI should know when it is morning, afternoon, or evening for the user and adjust its greetings and session summaries accordingly. Fourth, progressive persona complexity: new users should get a simpler, more guided MOXI experience that gradually introduces advanced capabilities as the user's expertise (tracked via XP and interaction patterns) grows. Fifth, multi-language persona: the system prompt instructs MOXI to respond in the same language the user writes in, but there is no explicit language configuration in the persona system, and the Arabic interface of the app suggests this should be more formally handled.

5.3 Persona System Score

5.0

Persona Engine

Clean CRUD, lacks depth

6. Agent System

Hunter, Analyst, Coach, Governor + 4 LLM agents + MOXI unified companion

The agent system is the most complex and problematic subsystem in the MOXI architecture. It exists in three separate implementations that are poorly integrated. The first is the four "original" LLM-powered agents defined in copilot/server/agents.ts: Market Analyst (SMC/ICT technical analysis), Risk Manager (position sizing and exposure), News Analyst (fundamental and macro analysis), and Strategy Builder (trading plans and routines). These agents have detailed, well-crafted system prompts that inject user context, market data, and memory. They are genuinely intelligent because they leverage LLM reasoning and are orchestrated by the Agent Orchestrator (agent-orchestrator.ts) which supports single-agent calls, keyword-based auto-selection, multi-agent consensus with synthesis, and streaming responses. This system is the most mature and functional part of the agent architecture.

The second implementation is the four "v4" agents introduced as part of the AI v4 generation: Hunter (smart money opportunity scoring in hunter.agent.ts), Analyst (behavioral analysis reports in analyst.agent.ts), Coach (trading psychology in coach.agent.ts), and Governor (risk assessment in governor.agent.ts). These are hardcoded mock implementations that return static or random responses with zero LLM integration. The Hunter agent generates scores using Math.random() based on input string length. The Coach agent checks if the trade amount exceeds $1000 to decide if the user is "on tilt." The Governor agent calculates a simple percentage and blocks trades above 20% portfolio allocation. The Analyst agent returns a fixed behavioral pattern string. These mock implementations are not just incomplete; they are actively misleading because they produce deterministic, meaningless outputs while pretending to be AI-driven decisions. The third implementation is MOXI itself, described as a "unified companion" that combines all four agent capabilities through prompt engineering rather than actual agent composition.

The debate engine in src/domains/debate/engine/debate.engine.ts uses four voting agents (Analyst, Strategist, RiskGuard, Contrarian) that are separate from both the copilot agents and the v4 agents. These debate agents produce deterministic votes based on analysis result properties (pattern type, confidence level, risk level) without any LLM reasoning. The votes are weighted by confidence and agent weight, summed per side (BULL/BEAR/NEUTRAL), and the winner is determined by highest total score. Consensus is declared if 3+ agents agree. The RiskGuard has a special veto power that can override the consensus if the risk level is HIGH. This produces predictable outcomes that don't reflect genuine deliberation. The system would be far more valuable if each debate agent were powered by an actual LLM call with different system prompts representing different analytical perspectives.

6.1 Agent Comparison Matrix

Agent	Type	Source	Capability	State
Market Analyst	LLM-powered	agents.ts	SMC/ICT analysis, order blocks, FVGs	Active, functional
Risk Manager	LLM-powered	agents.ts	Position sizing, R:R, exposure analysis	Active, functional
News Analyst	LLM-powered	agents.ts	Economic calendar, central banks, sentiment	Active, functional
Strategy Builder	LLM-powered	agents.ts	Trading plans, routines, psychology	Active, functional
Hunter (v4)	Hardcoded mock	hunter.agent.ts	Smart money scoring	Random output, no AI
Analyst (v4)	Hardcoded mock	analyst.agent.ts	Behavioral analysis	Static text, no AI
Coach (v4)	Hardcoded mock	coach.agent.ts	Trading psychology	Amount-based check, no AI
Governor (v4)	Hardcoded mock	governor.agent.ts	Risk assessment	Percentage math, no AI
MOXI	Prompt-based	moxi/prompt.ts	Unified companion, all capabilities	Active, prompt-engineered
Debate Agents	Deterministic	debate/agents/*.ts	Voting: BULL/BEAR/NEUTRAL	Heuristic rules, no AI

6.2 Agent System Scores

7.0	1.5	4.0	3.0
Original Agents	v4 Mock Agents	MOXI Unified	Debate Engine
4 LLM-powered, funct	*Hardcoded, meaning*	*Prompt-only composi*	*Deterministic voting*

6.3 Critical Agent Problems

ID	Priority	Problem	Impact	Risk	Files	Root Cause
AG-001	P0	v4 agents are hardcoded mocks returning static/random responses	Critical	High	copilot/server/*.agent.ts	No LLM calls in any v4 agent
AG-002	P0	Three separate agent systems are not unified	Critical	High	Multiple files	Original agents, v4 agents, debate agents
AG-003	P0	Debate engine uses deterministic voting, not LLM deliberation	High	Medium	debate/engine/	Votes based on analysis properties
AG-004	P1	MOXI has no actual agent composition capability	High	Medium	moxi/prompt.ts	Unified companion is prompt-only

ID	Priority	Problem	Impact	Risk	Files	Root Cause
AG-005	P1	No agent lifecycle management (init, warm-up, shutdown)	Medium	Medium	None exists	Agents lack lifecycle hooks
AG-006	P1	Auto-selection uses simple keyword matching	Medium	Low	copilot/server/agents.ts	Could use embedding similarity
AG-007	P2	Consensus synthesis uses extra LLM call (cost overhead)	Low	Low	agent-orchestrator.ts	5 LLM calls for consensus mode

7. Context Engine

Parallel data aggregation for MOXI system prompts

The context engine in src/domains/moxi/context-engine.ts is one of the best-implemented components in the MOXI architecture. It assembles real-time context for MOXI's system prompt by fetching data in parallel from nine different sources: user profile (from profiles table), active strategy (from user_strategies), active signal trackings (from signal_tracking), recent analyses (last 5 from analyses), daily signals (last 5 from daily_signals), watchlist items (from watchlist_items), active price alerts (from price_alerts), live market prices (via fetchPrices API), and upcoming economic events (via fetchEconomicCalendar). All nine queries execute concurrently via Promise.all, and the engine then augments prices with pairs from the user's watchlist and tracked signals that were not already in the popular pairs list. The memory context is loaded separately and non-critically, meaning MOXI works fine without memories.

The parallel data fetching is excellent for performance and follows a robust pattern where each data source is wrapped in its own async function with independent error handling. If the economic calendar fails, the other eight sources still succeed. If memory context fails, it is silently omitted. This defensive pattern ensures MOXI always has some context to work with, even when individual data sources are unavailable. The MoxiContext interface is well-typed with specific fields for each data type, and the engine carefully maps database rows to typed objects, using Pick to select only the needed fields from signal trackings. The price augmentation logic properly deduplicates pairs and merges the popular pairs with user-specific pairs. However, the context engine has several significant limitations. First, there is no token budget management. The formatted context can grow very large when a user has many active trackings, watchlist items, and analyses, potentially exceeding the LLM's context window or consuming excessive tokens. Second, there is no relevance filtering: all fetched data is included regardless of the user's question. If a user asks about EUR/USD specifically, they still get context about all their other pairs, signals, and events. Third, the economic calendar only fetches 3 days of events, which may miss important catalysts further out. Fourth, there is no caching, meaning every MOXI request triggers nine parallel database queries plus API calls for prices and calendar data.

7.1 Data Sources and Latency

Source	Backend	Latency	Notes
User Profile	Supabase (profiles)	Low	Single row by user_id
Active Strategy	Supabase (user_strategies)	Low	Single row, is_active=true

Source	Backend	Latency	Notes
Signal Trackings	Supabase (signal_tracking)	Low	Up to 20 rows, pending/active
Recent Analyses	Supabase (analyses)	Low	Last 5 analyses by user_id
Daily Signals	Supabase (daily_signals)	Low	Last 5 global signals
Watchlist	Supabase (watchlist_items)	Low	Up to 20 items
Price Alerts	Supabase (price_alerts)	Low	Up to 10 active alerts
Live Prices	fetchPrices API	Medium	Popular pairs + augmented
Economic Calendar	fetchEconomicCalendar	Medium	3-day forecast
User Memory	MemoryStore	Low	All categories for user_id

7.2 Context Engine Scores

6.0	3.0	4.0	5.0
Data Assembly	Token Manageme	Relevance	Caching
Parallel, resilient, well-	*No budget, no filteri*	*All data included regardless*	*No cache layer exists*

7.3 Context Engine Problems

ID	Priority	Problem	Impact	Risk	Files	Root Cause
CE-001	P0	No token budget management - context can exceed LLM limits	Critical	High	moxi/context-engine.ts, moxi/prompt.ts	No token counting or budget enforcement
CE-002	P1	No relevance filtering - all data included in every request	High	Medium	moxi/context-engine.ts	User question not used to filter context
CE-003	P1	No caching layer - 9+ DB queries per MOXI request	High	Medium	moxi/context-engine.ts	Every request is cold
CE-004	P1	Economic calendar limited to 3 days	Medium	Low	moxi/context-engine.ts	fetchEconomicCalendar(3) hardcoded
CE-005	P2	Price augmentation has sequential fallback fetch	Low	Low	moxi/context-engine.ts	Extra await fetchPrices after parallel

8. AI Workflows

Current workflow patterns: single request, tool execution, consensus

The MOXI system implements three primary workflow patterns. The first and most common is the single-request workflow, where a user sends a message to askMoxi, the system builds context, attempts tool execution via intent detection, and falls back to LLM response if no tool matches. This is a simple request-response pattern with no conversation state persistence. The second workflow is the consensus mode, available through the Agent Orchestrator, where all four LLM agents receive the same question independently and their responses are synthesized into a unified answer by a fifth LLM call. This produces comprehensive multi-perspective analysis but is expensive (5 LLM calls) and slow. The third workflow is the streaming mode, where the Agent Orchestrator yields chunks as the LLM generates them, providing progressive output to the frontend via SSE (Server-Sent Events).

The askMoxi workflow in functions.ts follows a specific sequence: (1) authenticate and rate-limit the request (25 requests per minute per user), (2) build context and load persona in parallel, (3) format context and load tool descriptions, (4) attempt P1 intelligence layer via copilot-agent (intent detection, tool dispatch), (5) if tool execution fails or no intent found, fall back to AI via LLMRouter with MOXI's system prompt, (6) return the response. This is a clean two-tier architecture (tool layer + AI fallback) but has critical gaps. There is no conversation persistence: every request starts fresh with only the explicitly provided history array (limited to 30 messages). There is no multi-step planning: MOXI cannot break complex requests into sequential tool calls. There is no agent collaboration: the four LLM agents never interact with MOXI. There is no background processing: MOXI cannot proactively run analyses or generate insights between user requests.

The notification workflow is the closest to a proactive AI workflow. It is triggered by the getMoxiInsights endpoint, which builds context, runs three detection algorithms (overexposure, signal proximity, event risk), sorts results by severity, and returns insights for the frontend to display. This could be extended into a push notification workflow where insights are generated on a schedule (e.g., every 15 minutes) and pushed via the notification channels (in-app, Telegram, email, webhook). However, this scheduled execution pattern does not yet exist. The daily signal generation is handled by a separate cron endpoint, but there is no equivalent cron for MOXI insights. The daily loop SQL migration (add_daily_loop.sql) suggests planned background processing, but the actual implementation is not yet connected to MOXI.

8.1 Workflow Comparison

Workflow	Entry Point	State	Pattern	Limitation
Single Request	askMoxi	Active	User message, no state	Fast, simple, no memory

Workflow	Entry Point	State	Pattern	Limitation
Tool Execution	copilot-agent	Active	Regex intent + dispatch	Brittle, no LLM understanding
Consensus	agent-orchestrator	Active	4 agents + synthesis	5 LLM calls, expensive
Streaming	agent-orchestrator	Active	SSE progressive output	Good UX, not used by MOXI
Proactive Insights	notification-hub	Active	On-demand, 3 detectors	No scheduled push
Daily Loop	daily_loop.sql	Schema only	Background processing	Migration exists, no code
Morning Brief	Missing	Not built	Scheduled daily summary	Not implemented
Multi-Step Plan	Missing	Not built	Complex request decomposition	Not implemented

8.2 AI Workflows Score

4.0	3.5	4.5
Workflow Coverage	Workflow Quality	Integration
3 of 8 patterns implemented	*Basic request-response only*	*Clean two-tier architecture*

9. Tool Calling

7 MOXI tools + ToolRegistry with keyword intent detection

The tool calling system has two layers: the MOXI tool definitions in src/domains/moxi/tools.ts (which define what MOXI can do conceptually) and the ToolRegistry in src/shared/tool-registry/ (which defines what can actually be executed). The MOXI tools define seven capabilities: analyzePair (run SMC/ICT analysis on a trading pair), trackSignal (convert analysis into a monitored signal), createPriceAlert (set price-based notifications), getMarketSummary (current market state), getPortfolio (user holdings and performance), getWatchlist (user watchlist with prices), and getSignalStatus (active signal tracking status). Each tool has a name, description, parameter schema with types and required flags, and a category for UI grouping.

The actual tool execution is handled by the ToolRegistry (src/shared/tool-registry/types.ts and bootstrap.ts), which registers tools with full parameter schemas, permission requirements, and handler functions. The ToolRouter (src/shared/tool-router/index.ts) dispatches tool calls by name with extracted parameters. The copilot-agent.ts bridges the gap between user messages and tool execution through regex-based intent detection. When a user message matches a pattern (e.g., "create alert for BTC above 110000"), the system extracts parameters using simple regex and string matching, validates required parameters, dispatches the tool through ToolRouter, and formats the result into a user-friendly response. This system works for straightforward requests but has fundamental limitations. The intent detection is regex-only with no semantic understanding. Parameter extraction is brittle (e.g., extractPair uses a hardcoded list of common pairs). There is no native LLM function calling integration, meaning the model cannot reason about which tools to use or how to combine them. There is no tool composition (chaining multiple tools in sequence) and no parallel tool execution.

The MOXI tool definitions and the ToolRegistry tools have different schemas and names, creating an impedance mismatch. The MOXI tools use camelCase names (analyzePair, createPriceAlert) while the registry tools may use different names (analyzeAsset, createAlert). The mapping between user intent, MOXI tool definition, and actual executable tool is not explicit, which could lead to confusion when extending the system. Additionally, the tool descriptions in the system prompt are generated by ToolRegistry.toolDescriptionsForPrompt(), which may not align with the descriptions in the MOXI tools definitions, creating potential confusion for the LLM about its actual capabilities.

9.1 Tool Registry Inventory

Tool Name	Category	Params	Description	State
analyzePair	analysis	pair, timeframe	SMC/ICT analysis on pair	Working
trackSignal	signals	analysisId	Convert analysis to signal	Working
createPriceAlert	alerts	pair, condition, price, note	Price-based notification	Working
getMarketSummary	data	(none)	Current market state	Working
getPortfolio	data	(none)	User holdings and PnL	Working
getWatchlist	data	(none)	Watchlist with prices	Working
getSignalStatus	signals	(none)	Active signal trackings	Working
createJournalEntry	data	content	Save trading note	Working
fetchSignals	signals	pair?	Daily signals	Working
analyzeAsset	analysis	pair, timeframe?	Asset state analysis	Working
listAlerts	alerts	pair?	User price alerts	Working
deleteAlert	alerts	(none)	Cancel an alert	Working
fetchPortfolio	data	(none)	Trade journal entries	Working
getAssetState	data	pair	Current price and change	Working

9.2 Tool Calling Problems

ID	Priority	Problem	Impact	Risk	Files	Root Cause
TC-001	P0	Intent detection is regex-only, no LLM-based understanding	Critical	High	copilot/server/copilot-agent.ts	detectIntent uses regex patterns
TC-002	P0	No native LLM function calling integration	Critical	High	shared/llm/, moxi/functions.ts	Providers support it but not used
TC-003	P1	Parameter extraction is brittle with hardcoded pairs	High	Medium	copilot/server/copilot-agent.ts	extractPair uses COMMON_PAIRS list

ID	Priority	Problem	Impact	Risk	Files	Root Cause
TC-004	P1	No tool composition or multi-step execution	High	Medium	copilot/server/copilot-agent.ts	Single tool per request only
TC-005	P1	MOXI tools and ToolRegistry have schema mismatch	Medium	Medium	moxi/tools.ts vs shared/tool-registry/	Different names and schemas
TC-006	P2	No tool result caching	Low	Low	shared/tool-registry/	Every call fetches fresh data

9.3 Tool Calling Score

5.5	4.0	3.0	6.0
Tool Registry	Intent Detection	Function Calling	Execution
Well-designed, 14 to	*Regex-only, brittle*	*Not implemented*	*Clean dispatch pattern*

10. Daily Operating System

MOXI as an always-on trading assistant with scheduled intelligence

The "Daily Operating System" concept envisions MOXI as an always-on trading assistant that proactively manages the user's trading day from pre-market analysis through end-of-day reflection. This chapter evaluates the current state of scheduled and background intelligence capabilities. The closest existing implementation is the daily signal generation system, which runs on a cron schedule to produce daily trading signals stored in the daily_signals table. The reanalysis cron endpoint (server/api/reanalysis-cron.ts) triggers re-analysis of existing positions. The check-alerts endpoint (server/api/check-alerts.ts) monitors price alerts and triggers notifications when conditions are met. These cron endpoints provide the foundation for a daily operating system, but they are not integrated with MOXI and do not provide the comprehensive daily experience the product vision describes.

The daily_loop SQL migration (add_daily_loop.sql) defines a table structure for daily loop execution, suggesting planned background processing that would track daily tasks, execution status, and results. However, the actual execution logic for this daily loop is not yet implemented. The experiment domain (src/domains/experiment/) has infrastructure for evolutionary prompt optimization, which could be connected to the daily operating system to continuously improve MOXI's performance. The copilot conversations module (copilot/conversations.ts) manages chat history but lacks the persistence and summarization needed for multi-session context. The missing components for a true daily operating system include: morning brief generation (a personalized pre-market summary delivered at a configurable time), mission system (daily trading tasks and goals based on the user's strategy and market conditions), portfolio health monitoring (continuous tracking of portfolio metrics with proactive alerts), goal setting and tracking (user-defined trading goals that MOXI monitors and reports on), end-of-day reflection (automated performance review and learning extraction), and scheduled insight delivery (pushing MOXI's proactive insights to notification channels on a schedule rather than on-demand).

The notification infrastructure (src/shared/notifications/) provides four delivery channels: in-app (real-time UI updates), Telegram (via webhook integration), email (for important alerts), and webhook (for external system integration). Each channel has its own implementation with proper error handling and formatting. This infrastructure is ready to support a daily operating system but is not yet connected to scheduled MOXI workflows. The notification-hub's three detectors (overexposure, signal proximity, event risk) could be triggered on a schedule and pushed through these channels, but currently they only run on-demand when the frontend calls getMoxiInsights. Enabling scheduled push notifications would be a high-impact, relatively low-effort improvement that transforms MOXI from a reactive chatbot into a proactive daily companion.

10.1 Daily OS Components

Component	State	Priority	Description	Location
Morning Brief	Not built	P1	Personalized pre-market summary	moxi/morning-brief.ts
Daily Signals	Active (cron)	P0	Automated signal generation	server/api/generate-signals.ts
Alert Monitoring	Active (cron)	P0	Price alert triggers	server/api/check-alerts.ts
Reanalysis	Active (cron)	P1	Position re-analysis	server/api/reanalysis-cron.ts
Portfolio Monitor	Not built	P1	Continuous portfolio tracking	New: moxi/portfolio-health.ts
Goal Tracking	Not built	P1	User-defined trading goals	New: moxi/goals.ts
Insight Push	On-demand only	P1	Scheduled proactive insights	moxi/notification-hub.ts
End-of-Day Review	Not built	P1	Automated reflection report	New: moxi/reflection.ts
Mission System	Not built	P2	Daily trading tasks/goals	New: moxi/missions.ts

10.2 Daily OS Score

3.5	6.0	3.0
Scheduled Intel	Infrastructure	Coverage
Only 3 of 9 components active	*4 notification channels ready*	*No morning brief, no reflection*

11. Competitor Analysis

How MOXI compares to AI trading assistants in the market

The AI trading assistant market has matured significantly with products like TradingView's AI features, Moralis Money, CoinCodex AI, and various Telegram trading bots offering AI-powered analysis. This chapter compares MOXI's current capabilities against these competitors to identify competitive gaps and advantages. MOXI's unique value proposition is its unified companion approach (combining analysis, risk, coaching, and hunting in one entity), its deep integration with user data (signals, trackings, analyses, watchlist, memories), and its proactive intelligence (notification hub). However, most competitors offer features that MOXI currently lacks: persistent conversation threads, semantic memory, native function calling, streaming responses, mobile push notifications, multi-language support, and automated strategy execution.

Against TradingView's AI features, MOXI has the advantage of deeper user data integration (TradingView does not know about the user's open positions, risk preferences, or past mistakes) but the disadvantage of no charting integration beyond basic context. TradingView provides real-time chart annotations and drawing tools, while MOXI's chart intelligence integration is limited to basic session context injection. Against Telegram trading bots, MOXI has the advantage of a full web application with rich UI but lacks the immediacy of Telegram-based push notifications for time-sensitive alerts. The Telegram webhook integration exists but is not used for MOXI insights. Against specialized AI platforms like Moralis Money (which focuses on smart money and on-chain analytics), MOXI's Hunter agent is a hardcoded mock that generates random scores, making it unable to compete on smart money analysis. This is a significant competitive gap because smart money tracking is a major selling point in the crypto trading space.

MOXI's debate engine (multi-agent consensus) is a genuinely unique feature that no major competitor offers. The concept of running four analytical perspectives in parallel and synthesizing them into a unified answer is architecturally sophisticated and could be a strong differentiator if the agents were powered by actual LLM reasoning rather than deterministic heuristics. The persona system (8 avatar variants, customizable personality) is also a differentiator, as most competitors offer a single AI personality. The notification hub's three proactive detectors (overexposure, signal proximity, event risk) provide value that many competitors do not offer. However, these advantages are undermined by the fundamental weaknesses in memory, conversation persistence, and agent intelligence. A trader who discovers that MOXI does not remember their previous conversation or that the "AI analysis" comes from hardcoded logic will quickly lose trust in the system.

11.1 Competitive Position Matrix

Feature	Position	Strength	Notes
Unified Companion	Unique	Strong	No competitor combines all roles
Persistent Memory	Below	Weak	Competitors have conversation history
Streaming Response	Below	Weak	MOXI is request-response only
Smart Money Analysis	Below	None	Hunter agent is hardcoded random
Multi-Agent Consensus	Leading	Strong	Unique debate + synthesis feature
Push Notifications	Below	Medium	Infrastructure exists, not connected
Chart Integration	Parity	Medium	Basic TradingView context
Persona Customization	Leading	Strong	8 variants + custom personality
Strategy Execution	Below	None	No automated trading capability
Multi-Language	Parity	Medium	Prompt instruction only, no formal i18n

12. AI Scoring

Scoring MOXI across 13 quality dimensions

This chapter provides a comprehensive scoring of the MOXI AI system across 13 dimensions that collectively measure the quality, intelligence, and maturity of the AI capabilities. Each dimension is scored on a 1-10 scale with a semantic assessment (Strong 7+, Moderate 4-6, Weak 1-3), a detailed justification, and specific improvement recommendations. The 13 dimensions cover the full spectrum of AI system quality from fundamental capabilities (context understanding, response quality) through advanced features (reasoning, planning, learning) to operational concerns (latency, reliability, cost). The overall weighted score is 5.2 out of 10, reflecting a system that has solid architectural foundations but significant gaps in AI intelligence and advanced capabilities.

12.1 Dimension Scores

ID	Dimension	Score	Rating	Justification
D01	Context Understanding	6.0	Moderate	Parallel context fetch, no relevance filtering
D02	Response Quality	5.5	Moderate	Good prompts, no quality scoring
D03	Conversation Memory	2.0	Weak	No persistence, each request is independent
D04	Proactive Intelligence	6.5	Strong	3 working detectors, scheduled push missing
D05	Personalization Depth	4.0	Moderate	5 memory categories, no semantic search
D06	Multi-Agent Reasoning	3.5	Weak	4 mock agents + 4 LLM agents, not unified
D07	Tool Execution	5.5	Moderate	14 tools, regex intent, no function calling
D08	Reasoning Depth	3.0	Weak	No chain-of-thought, no planning engine
D09	Learning Capability	3.0	Weak	No outcome tracking, no feedback loop
D10	Response Latency	5.0	Moderate	No streaming for MOXI, cold context fetch
D11	Error Resilience	7.0	Strong	4-provider fallback, defensive coding
D12	Cost Efficiency	5.0	Moderate	No token counting, no caching
D13	Data Privacy	6.5	Strong	Auth required, no external data leakage

12.2 Score Distribution

Rating	Count	Percentage	Dimensions
Strong (7-10)	3	23%	Proactive Intelligence, Error Resilience, Data Privacy

Rating	Count	Percentage	Dimensions
Moderate (4-6)	7	54%	Context, Response Quality, Personalization, Tools, Latency, Cost
Weak (1-3)	3	23%	Conversation Memory, Multi-Agent Reasoning, Learning, Reasoning

12.3 Overall AI Score

5.2	6.5	2.0
Overall AI System	Best Dimension	Worst Dimension
Weighted average of 13 dimensions	*Proactive Intelligence*	*Conversation Memory*

13. Top Problems and Improvements

47 problems identified with root cause analysis and improvement mapping

This chapter consolidates all problems identified across the previous chapters into a comprehensive problem registry with root cause analysis, impact assessment, risk levels, and mapped improvement tasks. The 47 problems are categorized by subsystem and prioritized using a three-tier system: P0 (Critical) for problems that fundamentally block the AI Operating System vision or represent broken functionality, P1 (High) for significant quality and capability gaps, and P2 (Medium) for advanced features and optimizations. Each problem is linked to a specific task ID in Chapter 14 where applicable, creating a traceable path from problem identification to implementation.

The most impactful P0 problems are: (1) No conversation persistence (AI-001), which means MOXI cannot maintain context across messages and users must repeat themselves, (2) Four mock agents returning random/hardcoded outputs (AI-008), which undermines the credibility of the multi-agent system, (3) No token budget management (AI-003), which can cause context overflow and token waste, (4) No native LLM function calling (AI-006), which forces brittle regex intent detection, and (5) No semantic memory (AI-007), which limits MOXI's ability to form deep understanding of individual traders. These five P0 problems represent the minimum viable improvements needed to transform MOXI from a chatbot into an AI companion.

13.1 P0 Critical Problems

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MOXI-001	P0	No conversation persistence across sessions	Critical	High	moxi/functions.ts, copilot/conversations.ts	No chat storage mechanism
MOXI-002	P0	No short-term memory buffer for multi-turn context	Critical	High	moxi/functions.ts	History passed by client only, not stored
MOXI-003	P0	No token budget management for LLM context	Critical	High	moxi/context-engine.ts, moxi/prompt.ts	No token counting exists
MOXI-004	P0	v4 agents are hardcoded mocks, no LLM	Critical	High	copilot/server/*.agent.ts	Random/static responses
MOXI-005	P0	Debate engine uses deterministic voting	High	Medium	debate/engine/debate.engine.ts	Heuristic rules, not LLM
MOXI-006	P0	No native LLM function calling	Critical	High	shared/llm/, moxi/functions.ts	Providers support but unused
MOXI-007	P0	Memory system has no semantic search	Critical	High	shared/memory/store.ts	Key-value only, no embeddings

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MOXI-008	P0	Three agent systems not unified	Critical	High	Multiple domains	Original + v4 + debate agents
MOXI-009	P0	MOXI not positioned as product center	Critical	High	Layout, routing, UI	Buried in copilot UI
MOXI-010	P0	No automated risk limit enforcement	Critical	High	governor.agent.ts	Governor is mock, no enforcement

13.2 P1 High Priority Problems

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MOXI-011	P1	MOXI endpoint lacks streaming support	High	Medium	moxi/functions.ts	Only request-response, no SSE
MOXI-012	P1	No context caching layer	High	Medium	moxi/context-engine.ts	9+ DB queries per request
MOXI-013	P1	No relevance filtering for context data	High	Medium	moxi/context-engine.ts	All data in every prompt
MOXI-014	P1	News data not in MOXI context	Medium	Low	moxi/context-engine.ts	Only economic events, no news
MOXI-015	P1	Journal/notes not in MOXI context	Medium	Low	moxi/context-engine.ts	Trading notes not fetched
MOXI-016	P1	Memory key matching bug in retrieve()	High	Medium	shared/memory/store.ts	Filters by category only
MOXI-017	P1	No memory TTL or expiration	Medium	Low	shared/memory/store.ts	Stale memories persist
MOXI-018	P1	No behavior tracking from interactions	High	Medium	shared/memory/	Only last_intent recorded
MOXI-019	P1	No adaptive persona adjustment	High	Medium	moxi/persona.ts	Static persona, no evolution
MOXI-020	P1	No timezone awareness	Medium	Low	moxi/persona.ts	No session timing logic
MOXI-021	P1	No tool composition engine	High	Medium	shared/tool-registry/	Single tool per request
MOXI-022	P1	No parallel tool execution	Medium	Low	shared/tool-registry/	Sequential execution only
MOXI-023	P1	No tool result caching	Medium	Low	shared/tool-registry/	Every call is cold
MOXI-024	P1	Debate agents not LLM-powered	High	Medium	debate/agents/*.ts	Deterministic voting only
MOXI-025	P1	No agent lifecycle management	Medium	Low	No file exists	No init/warm-up/shutdown
MOXI-026	P1	No scheduled insight push	High	Medium	moxi/notification-hub.ts	On-demand only

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MOXI-027	P1	No notification scheduling system	Medium	Medium	shared/notifications/	No scheduler integration
MOXI-028	P1	No morning brief generation	High	Medium	Not built	No scheduled daily summary

13.3 P2 Medium Priority Improvements

ID	Priority	Problem	Impact	Risk	Files	Root Cause
MOXI-029	P2	Add RAG knowledge base for trading concepts	Medium	Low	New: moxi/knowledge/	No knowledge retrieval system
MOXI-030	P2	Add chain-of-thought reasoning	High	Medium	moxi/prompt.ts	No structured reasoning prompts
MOXI-031	P2	Add response quality scoring	Medium	Low	New: moxi/quality.ts	No quality measurement
MOXI-032	P2	Add outcome tracking system	High	Medium	New: moxi/outcomes.ts	No prediction result tracking
MOXI-033	P2	Add A/B prompt testing framework	Medium	Low	New: moxi/experiments/	No experimentation system
MOXI-034	P2	Add emotion detection for adaptive tone	Medium	Medium	moxi/persona.ts	No sentiment analysis
MOXI-035	P2	Add goal setting and tracking	Medium	Low	New: moxi/goals.ts	No user-defined goals
MOXI-036	P2	Add achievement/gamification system	Low	Low	New: moxi/achievements.ts	No gamification
MOXI-037	P2	Add trading execution tools	High	High	moxi/tools.ts	No automated trading
MOXI-038	P2	Add multi-modal support (chart images)	Medium	Medium	moxi/functions.ts	Text-only responses

13.4 Problem Distribution

Priority	Count	Percentage	Description
P0 - Critical	10	21%	Must-fix: blocks AI OS vision entirely
P1 - High	18	38%	Should-fix: significant quality improvements
P2 - Medium	10	21%	Nice-to-have: advanced competitive features
Enhancement	9	19%	Polish and optimization opportunities
TOTAL	47	100%	Complete problem inventory

14. Implementation Tasks

AI-001 through AI-052 with full planning details

This chapter provides a complete implementation roadmap organized into four sprints, with 52 tasks covering all improvements identified in this audit. Each task includes a unique identifier (AI-001 through AI-052), a descriptive name, priority rating (P0/P1/P2), difficulty assessment (Low/Medium/High/Very High), expected impact (Critical/High/Medium/Low), dependencies on other tasks, affected files, estimated hours, and sprint assignment. Tasks are organized so that P0 critical tasks form Sprint 1, high-priority foundational tasks form Sprint 2, workflow and integration tasks form Sprint 3, and advanced intelligence features form Sprint 4. The total estimated effort is approximately 1062 hours across 52 tasks, which translates to roughly 26 weeks with a team of two dedicated engineers, or 13 weeks with four engineers.

14.1 Sprint 1: Foundation (P0 Critical)

ID	Task	Pri	Diff	Impact	Deps	Files	Hrs	Sprint
AI-001	Conversation Persistence	P0	Medium	Critical	None	moxi/functions.ts, copilot/conversations.ts, migration	24	Sprint 1
AI-002	Short-term Memory Buffer	P0	High	Critical	AI-001	new: shared/memory/buffer.ts	32	Sprint 1
AI-003	Token Budget Manager	P0	High	Critical	None	new: moxi/token-budget.ts, moxi/context-engine.ts	28	Sprint 1
AI-004	Context Compression	P0	High	Critical	AI-003	new: moxi/compress.ts, moxi/prompt.ts	24	Sprint 1
AI-005	Planning Engine v1	P0	Very High	Critical	AI-006	new: moxi/planning-engine.ts	40	Sprint 1
AI-006	Native Function Calling	P0	High	Critical	None	shared/llm/types.ts, moxi/functions.ts	32	Sprint 1
AI-007	Semantic Memory (pgvector)	P0	Very High	Critical	None	shared/memory/store.ts, migration	40	Sprint 1
AI-008	Unify Agent Systems	P0	High	Critical	None	copilot/server/agents.ts, copilot/server/*.agent.ts	36	Sprint 1
AI-009	MOXI as Product Center	P0	Very High	Critical	AI-001, AI-008	Multiple domains, routing, layout	48	Sprint 1
AI-010	Automated Risk Limits	P0	High	Critical	None	governor.agent.ts, moxi/notification-hub.ts	28	Sprint 1

14.2 Sprint 2: Core AI (P1 High)

ID	Task	Pri	Diff	Impact	Deps	Files	Hrs	Sprint
AI-011	MOXI Streaming Support	P1	Medium	High	AI-001	moxi/functions.ts, copilot-stream.ts	20	Sprint 2
AI-012	Context Caching Layer	P1	Medium	High	None	moxi/context-engine.ts, shared/cache.ts	16	Sprint 2
AI-013	Relevance Context Filtering	P1	High	High	AI-003	new: moxi/relevance.ts	24	Sprint 2
AI-014	News in MOXI Context	P1	Low	Medium	None	moxi/context-engine.ts	8	Sprint 2
AI-015	Journal Integration	P1	Low	Medium	None	moxi/context-engine.ts, notes/	12	Sprint 2
AI-016	Fix Memory Key Matching	P1	Low	High	None	shared/memory/store.ts	4	Sprint 2
AI-017	Memory TTL System	P1	Medium	Medium	None	shared/memory/store.ts, migration	12	Sprint 2
AI-018	Behavior Tracking Events	P1	Medium	High	AI-002	new: shared/memory/behavior-tracker.ts	20	Sprint 2
AI-019	Adaptive Persona v1	P1	High	High	AI-018	moxi/persona.ts	24	Sprint 2
AI-020	Timezone Awareness	P1	Low	Medium	None	moxi/persona.ts	8	Sprint 2
AI-021	Tool Composition Engine	P1	High	High	AI-006	new: shared/tool-registry/composition.ts	28	Sprint 2
AI-022	Parallel Tool Execution	P1	Medium	Medium	AI-006	shared/tool-registry/types.ts	16	Sprint 2
AI-023	Tool Result Caching	P1	Medium	Medium	AI-012	shared/tool-registry/	12	Sprint 2
AI-024	LLM Debate Agents	P1	High	High	AI-008	debate/agents/*.ts	28	Sprint 2
AI-025	Agent Lifecycle Framework	P1	High	High	AI-008	new: shared/agent-lifecycle/	24	Sprint 2

14.3 Sprint 3: Workflows (P1)

ID	Task	Pri	Diff	Impact	Deps	Files	Hrs	Sprint
AI-026	Morning Brief Generator	P1	Medium	Very High	AI-004	new: moxi/morning-brief.ts	20	Sprint 3
AI-027	Daily Mission System	P1	Medium	High	AI-005	new: moxi/missions.ts	24	Sprint 3
AI-028	Portfolio Health Monitor	P1	Medium	High	AI-010	new: moxi/portfolio-health.ts	20	Sprint 3
AI-029	Goal Setting and Tracking	P1	Medium	High	None	new: moxi/goals.ts, migration	20	Sprint 3
AI-030	Expand Notification Detectors	P1	Medium	High	AI-028	moxi/notification-hub.ts	24	Sprint 3
AI-031	Notification Scheduling	P1	Medium	Medium	AI-030	moxi/notification-hub.ts	16	Sprint 3
AI-032	Notification Preference Learning	P1	High	Medium	AI-030	moxi/notification-hub.ts	20	Sprint 3
AI-033	Agent Collaboration Protocol	P1	Very High	High	AI-025	new: shared/agent-collaboration/	32	Sprint 3
AI-034	Context Cache Optimization	P1	Medium	Medium	AI-012	moxi/context-engine.ts	12	Sprint 3
AI-035	End-of-Day Reflection	P1	Medium	High	AI-001	new: moxi/reflection.ts	16	Sprint 3

14.4 Sprint 4: Intelligence (P2)

ID	Task	Pri	Diff	Impact	Deps	Files	Hrs	Sprint
AI-036	Knowledge Base with RAG	P2	Very High	High	AI-007	new: moxi/knowledge/	36	Sprint 4
AI-037	Chain-of-Thought Reasoning	P2	High	High	AI-005	moxi/prompt.ts	24	Sprint 4
AI-038	Response Quality Scoring	P2	Medium	Medium	None	new: moxi/quality.ts	16	Sprint 4
AI-039	Outcome Tracking System	P2	Medium	High	AI-001	new: moxi/outcomes.ts	20	Sprint 4
AI-040	A/B Prompt Testing	P2	Medium	Medium	AI-038	new: moxi/experiments/	20	Sprint 4
AI-041	Emotion Detection	P2	High	Medium	AI-019	moxi/persona.ts	20	Sprint 4
AI-042	Learning Style Adaptation	P2	Medium	Low	AI-041	moxi/persona.ts	12	Sprint 4

ID	Task	Pri	Diff	Impact	Deps	Files	Hrs	Sprint
AI-043	Achievement System	P2	Medium	Medium	AI-029	new: moxi/achievements.ts	16	Sprint 4
AI-044	Habit Tracking	P2	Medium	Medium	AI-018	new: moxi/habits.ts	16	Sprint 4
AI-045	Trading Execution Tools	P2	High	High	AI-006	moxi/tools.ts	28	Sprint 4
AI-046	Multi-modal Support	P2	Very High	Medium	AI-011	moxi/functions.ts	32	Sprint 4
AI-047	LLM Agent Routing	P2	Medium	Medium	AI-008	copilot/server/agents.ts	16	Sprint 4
AI-048	Enterprise Audit Logging	P2	Low	Low	AI-001	new: shared/audit/	12	Sprint 4
AI-049	MOXI Response Analytics	P2	Medium	Medium	AI-038	new: moxi/analytics/	16	Sprint 4
AI-050	Recommendation Engine	P2	High	High	AI-007	new: moxi/recommendations.ts	24	Sprint 4
AI-051	Self-Verification System	P2	High	Medium	AI-037	new: moxi/verification.ts	20	Sprint 4
AI-052	Drawdown Monitoring	P2	Medium	High	AI-028	moxi/notification-hub.ts	16	Sprint 4

14.5 Effort Summary by Sprint

Sprint	Tasks	Hours	Focus Areas
Sprint 1: Foundation (P0)	10	332	Memory, context, planning, function calling, agents, product
Sprint 2: Core AI (P1)	15	276	Streaming, caching, behavior, persona, tools, debate, lifecycle
Sprint 3: Workflows (P1)	10	204	Morning brief, missions, portfolio, goals, notifications, reflection
Sprint 4: Intelligence (P2)	17	302	RAG, reasoning, quality, outcomes, emotion, achievements, execution
TOTAL	52	1062	Approximately 26 weeks with 2 dedicated engineers

Executive Recommendation: Sprint 1 is the critical path. The 10 P0 tasks (332 hours) form the absolute minimum investment needed to transform MOXI from a capable chatbot into a credible AI companion. Without conversation persistence, semantic memory, token management, and native function calling, no other improvement will have meaningful impact. Sprint 1 should be staffed with the strongest available engineers and treated as a "make or break" investment. Sprint 2 and Sprint 3 can proceed in parallel streams after Sprint 1 completes. Sprint 4 is aspirational and should only be committed after the first three sprints demonstrate measurable improvements in user engagement and AI quality metrics.

--- End of MOXI AI Architecture Audit ---